



Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации

Ордена Трудового Красного Знамени федерального государственного
бюджетного образовательного учреждения высшего образования

«Московский технический университет связи и информатики»

Волго-Вятский филиал

Кафедра инфокоммуникационных и общепрофессиональных дисциплин (ИКиОПД)

**Проектирование программного обеспечения «ВетКартотека - система
управления карточками пациентов ветеринарной клиники» для
использования на занятиях по дисциплине «Технология разработки
программного обеспечения»**

Курсовая работа

*По дисциплине «Технология разработки программного обеспечения» специальности
09.02.03 «Информационные системы и программирование»*

Преподаватель:

Полтавец Е.В.

Выполнил студент

группы ИС-23

Шипицын Даниил Владимирович

г. Нижний Новгород

2026

Содержание

1 Введение	3
1.1 Цель и задачи работы	3
1.2 Место продукта среди аналогов	3
1.3 Ожидаемый результат	4
1.4 Структура работы	4
1.5 Используемые технологии	4
1.6 Ограничения прототипа	5
2 Теория	5
2.1 Архитектура настольных приложений на Python	5
2.2 Диаграммы, используемые при проектировании	6
2.3 Хранение данных в формате JSON	7
2.4 Валидация пользовательского ввода	7
2.5 Кастомные виджеты Tkinter	8
2.6 Управление состоянием приложения	8
2.7 Паттерн Observer через trace	9
2.8 Производительность и масштабируемость	9
3 Практика	9
3.1 Этап 1. Подготовка среды разработки	10
3.2 Этап 2. Архитектура приложения	10
3.3 Этап 3. Хранилище данных	11
3.4 Этап 4. Кастомные виджеты	11
3.5 Этап 5. Реализация главного класса VetClinicApp	11
3.6 Тестирование	12
3.7 Функциональные требования и их реализация	13
3.8 Примеры использования	14
4 Заключение	15

1 Введение

В рамках курсовой работы разработан программный модуль **ВетКартотека** - настольное приложение для управления карточками пациентов ветеринарной клиники. Небольшие ветеринарные кабинеты зачастую вынуждены вести учёт животных вручную - в бумажных журналах или таблицах Excel, не имея доступного специализированного инструмента. Существующие решения - либо громоздкие платные CRM-системы, либо разрозненные таблицы. ВетКартотека занимает нишу лёгкого автономного инструмента: пользователь добавляет пациента, заполняет карточку и сохраняет - всё, без дополнительной инфраструктуры и подключения к интернету.

1.1 Цель и задачи работы

Цель курсовой работы - разработать удобное графическое приложение, позволяющее вести учёт пациентов ветеринарной клиники: добавлять, редактировать, удалять карточки животных с медицинской информацией, а также осуществлять поиск и фильтрацию по виду животного.

Для достижения поставленной цели были определены следующие задачи:

- спроектировать и реализовать графический интерфейс приложения с тёмной темой оформления;
- реализовать хранение данных в формате JSON с поддержкой чтения и записи;
- обеспечить создание, редактирование и удаление карточек пациентов;
- реализовать поиск по кличке и ФИО владельца, а также фильтрацию по виду животного;
- предусмотреть валидацию пользовательского ввода и вывод информативных сообщений об ошибках.

1.2 Место продукта среди аналогов

На сегодняшний день существует несколько категорий инструментов для учёта пациентов ветеринарных клиник. Во-первых, это профессиональные ветеринарные информационные системы (ВетМенеджер, VetDesk и др.) - функциональные, но дорогостоящие и требующие обучения персонала. Во-вторых, онлайн-сервисы управления записями клиентов - удобны, но требуют постоянного интернет-соединения и регистрации. В-

третьих, электронные таблицы Excel или Google Sheets - бесплатны, однако не имеют специализированного интерфейса и неудобны при росте базы данных.

ВетКартотека занимает промежуточную нишу: это самостоятельное приложение с графическим интерфейсом, не требует ни интернета, ни платной подписки, ни знания программирования от пользователя. Запускается одной командой, выглядит современно и выполняет конкретную задачу - учёт пациентов ветеринарной клиники.

1.3 Ожидаемый результат

По итогам работы пользователь должен иметь возможность за несколько секунд найти любого пациента по кличке или имени владельца, просмотреть или отредактировать его карточку, а также добавить нового пациента. При ошибке ввода (например, пустая кличка) программа должна выводить понятное предупреждение без аварийного завершения. Это базовое требование к надёжности пользовательского интерфейса, реализованное через диалоги `tkinter.messagebox`.

1.4 Структура работы

Работа построена по классической схеме: постановка задачи - теоретическая основа - реализация - проверка и выводы. В разделе «Введение» описывается предметная область и мотивация. В разделе «Теория» излагаются программные концепции, на которых основана реализация. В разделе «Практика» описывается поэтапный процесс разработки и тестирования. В разделе «Заключение» подводятся итоги и намечаются направления дальнейшего развития.

1.5 Используемые технологии

В качестве языка реализации был выбран Python, поскольку он хорошо известен, имеет богатую экосистему библиотек для разработки GUI-приложений и работы с данными, а также позволяет быстро получить рабочий прототип. Основные используемые модули и библиотеки:

- `tkinter` - стандартная библиотека Python для создания графических интерфейсов; используется как основной фреймворк приложения;
- `ttk` - расширенный набор виджетов `tkinter` с поддержкой тем оформления;

- json - стандартный модуль для сериализации и десериализации данных в формате JSON;
- os - модуль для работы с файловой системой (проверка существования файла данных);
- datetime - модуль для получения текущей даты и времени при создании записи;
- re - модуль регулярных выражений для валидации телефонного номера и e-mail.

1.6 Ограничения прототипа

Поскольку работа носит учебный характер, приложение не претендует на полноту профессиональных ветеринарных систем. В текущей версии не поддерживаются: история визитов, прикрепление фотографий животных, многопользовательский режим, резервное копирование и экспорт данных в другие форматы. Эти ограничения осознанны: задача состояла в том, чтобы сделать простой и понятный инструмент, который хорошо справляется с одной задачей - ведением карточек пациентов ветеринарной клиники.

2 Теория

2.1 Архитектура настольных приложений на Python

Настольные приложения на Python, как правило, строятся по одному из двух подходов: процедурному или объектно-ориентированному. В процедурном подходе все функции и переменные существуют в глобальном пространстве имён, что затрудняет сопровождение при росте кодовой базы. Объектно-ориентированный подход позволяет инкапсулировать состояние приложения и связанную с ним логику внутри класса.

В приложении ВетКартотека применяется ООП: главный класс VetClinicApp наследуется от tk.Tk (корневое окно Tkinter). Это стандартный паттерн для Tkinter-приложений - весь интерфейс и все обработчики событий являются методами одного класса, что обеспечивает чистое разделение отображения и логики.

Для создания кастомных визуальных элементов используется наследование от базовых виджетов (tk.Canvas, tk.Frame). Класс RoundedFrame реализует карточку с закруглёнными углами путём отрисовки многоугольника через Canvas. Класс ModernEntry создаёт поле ввода

с поддержкой placeholder-текста и анимацией рамки при фокусировке. Класс PurpleButton реализует интерактивную кнопку с эффектом наведения.

2.2 Диаграммы, используемые при проектировании

Для формализации требований к разрабатываемому программному обеспечению, анализа предметной области и документирования архитектуры приложения «ВетКартотека» были применены следующие типы диаграмм, соответствующие методологиям функционального и объектно-ориентированного моделирования.

Диаграмма функционального моделирования в нотации IDEF0 позволяет представить систему как совокупность взаимосвязанных функций. В контексте курсового проекта IDEF0-модель используется для:

- выявления основных процессов: ввод данных о животном и владельце, сохранение в JSON-файл, поиск и фильтрация записей, отображение карточки, удаление записи;
- определения входных данных (ключика, вид, порода, ФИО владельца, медицинские данные), управляющих воздействий (методические указания, требования к надёжности), механизмов (Python, tkinter, json) и выходных результатов (заполненная карточка, сохранённый файл);
- декомпозиции сложного процесса ведения картотеки на более простые подпроцессы, что облегчает последующее написание кода.

Диаграмма потоков данных (DFD) дополняет IDEF0, акцентируя внимание на перемещении данных между источниками, процессами и накопителями. При разработке приложения «ВетКартотека» DFD необходима для:

- отслеживания преобразования пользовательского ввода в структуру словаря Python и последующей записи в JSON-файл;
- описания взаимодействия пользователя с программными компонентами: ввод данных → валидация → сохранение → обновление списка → отображение карточки;
- выявления точек временного хранения данных (список patients в памяти как кэш между обращениями к файлу).

Благодаря DFD удалось уточнить интерфейсы между модулями программы (хранилище, форма ввода, список пациентов) и минимизировать избыточные операции чтения файла.

Диаграмма кооперации иллюстрирует взаимодействие объектов в рантайме: она показывает, какие объекты обмениваются сообщениями при выполнении типовых сценариев. Например, сценарий «сохранение карточки»: пользователь нажимает кнопку → `VetClinicApp._save()` вызывает `_collect_form()` → получает словарь → вызывает `save_patients()` → обновляет список через `_refresh_list()` → выводит тост-уведомление. Это помогает реализовать шаблон проектирования MVC при написании кода.

Диаграмма развёртывания описывает физическое размещение компонентов программного обеспечения на аппаратных узлах. Для разрабатываемого приложения она показывает клиентское рабочее место (ПК администратора клиники), на котором установлены интерпретатор Python 3.x и файл `patients.json` как локальное хранилище данных. В случае сетевой версии возможно размещение файла на сетевом диске, однако текущая версия является полностью автономной.

2.3 Хранение данных в формате JSON

JSON (JavaScript Object Notation) - текстовый формат обмена данными, основанный на подмножестве синтаксиса JavaScript. В Python работа с JSON обеспечивается стандартным модулем `json`, не требующим установки дополнительных пакетов.

В приложении ВетКартотека данные хранятся в файле `patients.json` как массив объектов. Каждый объект соответствует одному пациенту и содержит поля: `id` (уникальный идентификатор, сформированный из временной метки), `name`, `species`, `breed`, `birth`, `sex`, `color`, `chip`, `owner`, `phone`, `email`, `address`, `vaccine`, `deworm`, `allergies`, `chronic`, `notes`, `created`.

Функция `load_patients()` читает файл при запуске приложения и возвращает список словарей Python. Функция `save_patients()` сериализует список обратно в JSON с отступами (`indent=2`) и поддержкой кириллицы (`ensure_ascii=False`). Такой подход обеспечивает человекочитаемый файл данных, который можно открыть и при необходимости отредактировать в любом текстовом редакторе.

2.4 Валидация пользовательского ввода

Валидация ввода - обязательная часть любого приложения, работающего с данными. Некорректный ввод может привести как к ошибкам при сохранении, так и к появлению «мусорных» записей в базе данных.

В приложении ВетКартотека реализованы следующие проверки. Во-первых, проверка обязательных полей: перед сохранением метод `_save()` проверяет, заполнены ли поля «Кличка» и «ФИО владельца»; при пустом значении выводится диалог `messagebox.showwarning` и сохранение прерывается. Во-вторых, поддержка placeholder-текста в полях ввода: класс `ModernEntry` возвращает пустую строку, если в поле содержится только текст-подсказка, тем самым не допуская сохранения служебного текста как реальных данных.

Уникальный идентификатор записи формируется автоматически из текущей временной метки с точностью до микросекунды (`datetime.now().strftime("%Y%m%d%H%M%S%f")`), что практически исключает коллизии при создании нескольких записей подряд.

2.5 Кастомные виджеты Tkinter

Стандартные виджеты Tkinter имеют устаревший внешний вид, не соответствующий современным требованиям к дизайну приложений. Для создания современного интерфейса с тёмной темой в приложении ВетКартотека реализованы три кастомных виджета путём наследования от базовых классов.

RoundedFrame - виджет-контейнер на основе `tk.Canvas`. Отрисовывает закруглённый прямоугольник как многоугольник с параметром `smooth=True`. Обрабатывает событие `<Configure>` для перерисовки при изменении размеров окна. Используется как «карточка» для визуального группирования элементов интерфейса.

ModernEntry - поле ввода с поддержкой placeholder-текста и эффекта фокуса. При получении фокуса рамка меняет цвет на акцентный (фиолетовый) и становится шире. Placeholder автоматически скрывается при вводе и восстанавливается при очистке поля. Метод `get()` возвращает пустую строку вместо текста placeholder.

PurpleButton - интерактивная кнопка на основе `tk.Canvas` с тремя стилями: `primary` (фиолетовый фон), `danger` (красный фон для кнопки удаления) и `ghost` (прозрачный фон с рамкой). Поддерживает эффект наведения курсора (`hover`) через обработчики событий `<Enter>` и `<Leave>`.

2.6 Управление состоянием приложения

Состояние приложения ВетКартотека включает три ключевых элемента: список пациентов `patients` (загружается из файла при запуске и обновляется при каждой операции

сохранения/удаления), идентификатор выбранного пациента `selected_id` (None при работе с новой записью) и строка поискового запроса `search_var` (StringVar, отслеживаемая через trace).

При выборе пациента в списке метод `_on_select()` считывает идентификатор из вспомогательного списка `_list_ids` (параллельный список ID, соответствующих строкам Listbox) и заполняет форму методом `_fill_form()`. Это позволяет избежать хранения данных непосредственно в виджетах и сохранять чистое разделение данных и представления.

2.7 Паттерн Observer через trace

Для реализации мгновенного поиска (без кнопки «Найти») используется механизм trace переменных Tkinter. Метод `search_var.trace("w", callback)` регистрирует функцию обратного вызова, которая вызывается при каждом изменении содержимого строки поиска. Callback вызывает `_refresh_list()`, которая фильтрует список пациентов по текущему запросу и виду животного, и обновляет Listbox.

Это реализация паттерна Observer (Наблюдатель): StringVar является субъектом, а метод `_refresh_list` - наблюдателем. Такой подход обеспечивает реактивное поведение интерфейса без дополнительного кода опроса состояния.

2.8 Производительность и масштабируемость

При текущей реализации весь список пациентов хранится в оперативной памяти. Для типичной ветеринарной клиники с несколькими сотнями или тысячами пациентов это не создаёт проблем: файл `patients.json` объёмом в тысячу записей займёт около 200-500 КБ, а операция фильтрации линейной сложности $O(n)$ выполняется практически мгновенно. При дальнейшем росте базы целесообразен переход на SQLite, что потребует минимальных изменений в методах `load_patients()` и `save_patients()`.

3 Практика

Практическая часть работы заключалась в поэтапной разработке приложения ВетКартотека с нуля. Ниже описан процесс реализации каждого компонента.

3.1 Этап 1. Подготовка среды разработки

Первым шагом была подготовка рабочей среды. Для изоляции зависимостей проекта было создано виртуальное окружение:

```
python -m venv venv
venv\Scripts\activate # Windows
source venv/bin/activate # Linux/macOS
```

Библиотека `tkinter` входит в стандартную поставку Python и не требует отдельной установки. Используемые при разработке версии: Python 3.11, `tkinter` 8.6. Дополнительных пакетов устанавливать не требуется - все импортируемые модули (`json`, `os`, `datetime`, `re`) являются частью стандартной библиотеки.

3.2 Этап 2. Архитектура приложения

Приложение реализовано в объектно-ориентированном стиле. Центральным элементом является класс `VetClinicApp`, наследуемый от `tk.Tk` (главное окно Tkinter). Такой подход позволяет инкапсулировать всё состояние приложения и все методы внутри одного класса.

Структурно модуль включает следующие компоненты:

- Глобальные константы цветовой палитры (`BG_DARK`, `PURPLE_MAIN` и др.) - определены в начале файла для единого управления темой оформления;
- Функции хранилища `load_patients()` и `save_patients()` - отвечают за чтение и запись JSON-файла;
- Кастомные виджеты `RoundedFrame`, `ModernEntry`, `PurpleButton` - визуальные компоненты многократного использования;
- Класс `VetClinicApp` - главный класс приложения, содержащий методы построения интерфейса (`_build_ui`, `_build_left`, `_build_right`, `_build_form`) и методы бизнес-логики (`_save`, `_delete`, `_refresh_list` и др.).

3.3 Этап 3. Хранилище данных

Функция `load_patients()` проверяет существование файла `patients.json` через `os.path.exists()` и, если файл присутствует, читает его содержимое через `json.load()`. При

отсутствии файла возвращается пустой список - это позволяет приложению корректно запускаться при первом использовании без какой-либо ручной инициализации.

Функция `save_patients()` записывает текущий список пациентов в файл через `json.dump()` с параметрами `ensure_ascii=False` (поддержка кириллицы) и `indent=2` (человекочитаемое форматирование). Файл перезаписывается целиком при каждом сохранении - это простейшая и надёжная стратегия для небольших объёмов данных.

3.4 Этап 4. Кастомные виджеты

Реализация кастомных виджетов потребовала глубокого понимания событийной модели Tkinter. Класс `ModernEntry` использует `Canvas` как контейнер для поля `Entry`, что позволяет отрисовывать произвольную форму рамки. Событие `<Configure>` перехватывается для обновления ширины поля `Entry` при изменении размеров окна через `itemconfig`.

Класс `PurpleButton` реализует все три визуальных стиля через единый метод `_draw()`, который выбирает цвета в зависимости от значений атрибутов `_style` и `_hover`. Метод вызывается при инициализации, наведении и снятии курсора, а также при изменении размеров - это обеспечивает корректное отображение кнопки в любых условиях.

3.5 Этап 5. Реализация главного класса `VetClinicApp`

Метод `_build_ui()` создаёт общую компоновку: заголовок (`header`), левую панель со списком пациентов и правую панель с формой карточки. Левая панель включает строку поиска, фильтры по виду животного и список `Listbox` с прокруткой. Правая панель содержит прокручиваемую форму с группами полей: данные животного, данные владельца, медицинская информация и примечания.

Метод `_save()` выполняет следующую последовательность: считывает данные из формы через `_collect_form()`, проверяет обязательные поля, генерирует идентификатор для новой записи или обновляет существующую, вызывает `save_patients()` и обновляет список. В конце отображается тост-уведомление через `_show_toast()` с автоматическим исчезновением через 2 секунды (метод `after(2000, toast.destroy)`).

Метод `_delete()` запрашивает подтверждение через `messagebox.askyesno()`, удаляет запись из списка в памяти через `list comprehension` и сохраняет обновлённый список. После удаления форма очищается, список обновляется, статусный индикатор возвращается в неактивное состояние.

3.6 Тестирование

После завершения реализации было проведено ручное тестирование по набору типовых сценариев. Результаты сведены в таблицу 1.

Сценарий	Ожидаемый и фактический результат
Запуск без файла patients.json	Приложение запускается с пустым списком - подтверждено
Добавление нового пациента	Запись сохраняется, появляется в списке - подтверждено
Сохранение с пустой кличкой	Диалог предупреждения - выведен
Сохранение с пустым ФИО владельца	Диалог предупреждения - выведен
Поиск по кличке	Список фильтруется в реальном времени - подтверждено
Поиск по ФИО владельца	Список фильтруется в реальном времени - подтверждено
Фильтр по виду животного	Отображаются только животные выбранного вида - подтверждено
Редактирование существующей карточки	Данные обновляются, ID и дата создания сохраняются - подтверждено
Удаление пациента с подтверждением	Запись удаляется из списка и файла - подтверждено
Отмена удаления	Запись сохраняется - подтверждено
Оператор «Очистить» форму	Все поля сбрасываются до placeholder - подтверждено
Повторный запуск после сохранения	Все записи корректно загружены из файла - подтверждено

Все двенадцать сценариев завершились ожидаемым результатом. Дополнительно проверялось поведение при добавлении нескольких пациентов подряд с последующим поиском - в всех случаях список обновлялся корректно.

3.7 Функциональные требования и их реализация

Ниже приведена расширенная таблица функциональных требований с указанием соответствующих компонентов реализации. Она позволяет убедиться в полноте покрытия всех заявленных возможностей.

Таблица 2: Функциональные требования и их реализация

ID	Описание требования
FR-1	Поле ввода кличек животного; обязательное поле с валидацией при сохранении.
FR-2	Поля ввода вида, породы, даты рождения, пола, окраса, номера чипа.
FR-3	Поля ввода ФИО владельца, телефона, e-mail, адреса; ФИО обязательно.
FR-4	Поля ввода медицинской информации: вакцинация, дегельминтизация, аллергии, хронические болезни.
FR-5	Многострочное поле примечаний (tk.Text) с поддержкой переноса строк.
FR-6	Сохранение данных в JSON-файл с поддержкой кириллицы.
FR-7	Загрузка данных из JSON-файла при запуске; корректная работа при отсутствии файла.
FR-8	Уникальный идентификатор записи на основе временной метки.
FR-9	Автоматическая фиксация даты и времени создания карточки.
FR-10	Поиск по кличке и ФИО владельца в реальном времени через trace.
FR-11	Фильтрация по виду животного (Все / Собака / Кошка / Птица / Грызун / Другое).
FR-12	Предупреждение при попытке сохранить запись с пустой кличкой или ФИО.
FR-13	Подтверждение перед удалением записи через messagebox.askyesno.
FR-14	Очистка формы кнопкой «Очистить» с восстановлением placeholder-текста.
FR-15	Тост-уведомление об успешном сохранении или удалении (исчезает через 2 с).
FR-16	Статусный индикатор (цветная точка) состояния выбранной записи.
FR-17	Счётчик общего числа пациентов в заголовке приложения.
FR-18	Иконки видов животных (эмодзи) в строках списка для быстрого визуального распознавания.

3.8 Примеры использования

Для наглядной демонстрации функциональных возможностей разработанного приложения «ВетКартотека» и оценки его пригодности для использования в учебном процессе

по дисциплине «Информатика» были подготовлены следующие сценарии работы. Каждый из них иллюстрирует ключевой аспект взаимодействия пользователя с программой.

Главный экран приложения. При запуске отображается двухпанельный интерфейс. Левая панель содержит строку поиска, фильтры по виду животного и список пациентов. Правая панель содержит форму карточки пациента с полями, сгруппированными в три раздела: «Данные животного», «Данные владельца» и «Медицинская информация». В нижней части формы расположены кнопки действий: «Сохранить», «Очистить» и «Удалить». Тёмная цветовая схема с фиолетовым акцентом обеспечивает комфортную работу при длительном использовании.

Добавление нового пациента. Пользователь нажимает кнопку «+ Новый пациент» - форма очищается, заголовок меняется на «Новый пациент», статусный индикатор подсвечивается. Пользователь заполняет поля и нажимает «Сохранить». Если кличка и ФИО владельца заполнены - запись добавляется в список, в левой панели появляется новая строка с соответствующей иконкой вида животного, счётчик пациентов в заголовке увеличивается, отображается тост-уведомление «✓ Сохранено успешно».

Поиск и фильтрация. Пользователь начинает вводить кличку или фамилию владельца в строку поиска - список пациентов мгновенно фильтруется, не дожидаясь нажатия Enter. Одновременно можно применить фильтр по виду животного, нажав на соответствующую кнопку над списком. Оба фильтра работают совместно: отображаются только записи, удовлетворяющие обоим условиям одновременно.

Обработка ошибочного ввода. При попытке сохранить карточку с незаполненной кличкой программа выводит диалоговое окно с предупреждением «Введите кличку животного!» и не выполняет сохранение. Аналогичное предупреждение выводится при пустом поле ФИО владельца. Данные, уже введённые в форму, при этом не теряются - пользователь может исправить ошибку и повторить сохранение.

4 Заключение

В ходе выполнения курсовой работы был разработан и протестирован программный модуль ВетКартотека - настольное приложение для управления карточками пациентов ветеринарной клиники.

Поставленная цель достигнута в полном объёме:

- реализован удобный двухпанельный графический интерфейс с тёмной темой оформления и кастомными виджетами;

- обеспечено хранение данных в формате JSON с поддержкой кириллицы;
- реализованы операции создания, редактирования и удаления карточек пациентов;
- реализован мгновенный поиск по кличке и ФИО владельца, а также фильтрация по виду животного;
- предусмотрена валидация обязательных полей с выводом информативных предупреждений.

Все 18 функциональных требований, сформулированных в начале работы, реализованы и подтверждены в ходе ручного тестирования по 12 сценариям.

В процессе разработки возникли несколько нетривиальных моментов. Во-первых, создание кастомных виджетов на основе Canvas потребовало понимания механизма перерисовки при событии <Configure>: без обработчика этого события закруглённые рамки не масштабировались бы при изменении размеров окна. Во-вторых, реализация placeholder-текста в ModernEntry потребовала аккуратной обработки состояний фокуса, чтобы текст-подсказка не попадал в сохраняемые данные. В-третьих, корректная работа параллельного списка `_list_ids` - это тонкий аспект синхронизации Listbox с данными, который легко упустить при первой реализации.

Разработанное приложение может использоваться в учебных целях на занятиях по информатике для демонстрации принципов ООП, работы с файлами, событийного программирования и проектирования GUI-приложений. Преимущество перед онлайн-сервисами состоит в полной автономности: приложение работает без интернета и не требует регистрации или специальных знаний от пользователя.

В рамках дальнейшего развития проекта можно выделить несколько направлений:

- добавить историю визитов для каждого пациента с датой, описанием приёма и назначениями;
- реализовать прикрепление фотографии животного к карточке;
- добавить экспорт данных в форматы CSV и PDF для печатных отчётов;
- реализовать резервное копирование файла данных с версионированием;
- перейти на SQLite для хранения данных при значительном росте базы пациентов;
- добавить автоматические модульные тесты для методов `_save`, `_delete` и `_refresh_list`.

